Chapter 3
Object Orientation

Shimels D(MPH/HI)

Objectives

- At the end of this chapter students should be able to:
- Understand **inheritance** and how to use it to develop new classes based on existing classes.
- Learn the notions of **superclasses** and **subclasses** and the relationship between them.
- Use keyword **extends** to create a class that inherits attributes and behaviors from another class.
- Hide member data of superclass -- **encapsulation**
- Access superclass members with **super** keyword from a subclass.
- Learn how **constructors** are used in inheritance hierarchies.
- Discover **polymorphism** and **dynamic binding**.
- Learn how **polymorphism** makes systems **extensible** and **maintainable**.

Objectives…

- Describe **casting** and explain why **explicit downcasting** is necessary.
- Use **final** classes and methods.
- Use **overridden methods** to effect polymorphism.
- Distinguish differences between **method overriding** and **method overloading**.
- Design and use **abstract classes** and **abstract methods**.
- Declare and implement **interfaces** and define classes that implement interfaces.
- Explore the similarities and differences among **concrete classes**, **abstract classes**, and **interfaces**.
- Learn about the methods of class **Object**, the direct or indirect superclass of all classes.

Inheritance

- A form of **software reuse** in which a new class is created by reusing an existing class's members and embellishing them with new or modified capabilities.
- The Java class hierarchy begins with class Object (in package java.lang):
  - *Every class in Java directly or indirectly extends (or "inherits from") Object.*
- Inheritance reprsents **is-a** relationship.
- Is-a represents **inheritance** where an object of a subclass can also be treated as an object of its superclass.

Example: Studnet is a Person.

Superclasses and Subclasses

- Superclasses tend to be “more general” and subclasses “more specific.”
- Because every subclass object is an object of its superclass, and one superclass can have many subclasses, the set of objects represented by a superclass is typically larger than the set of objects represented by any of its subclasses.

Inheritance examples:

Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Employee	Faculty, Staff

CommunityMember Inheritance Hierarchy

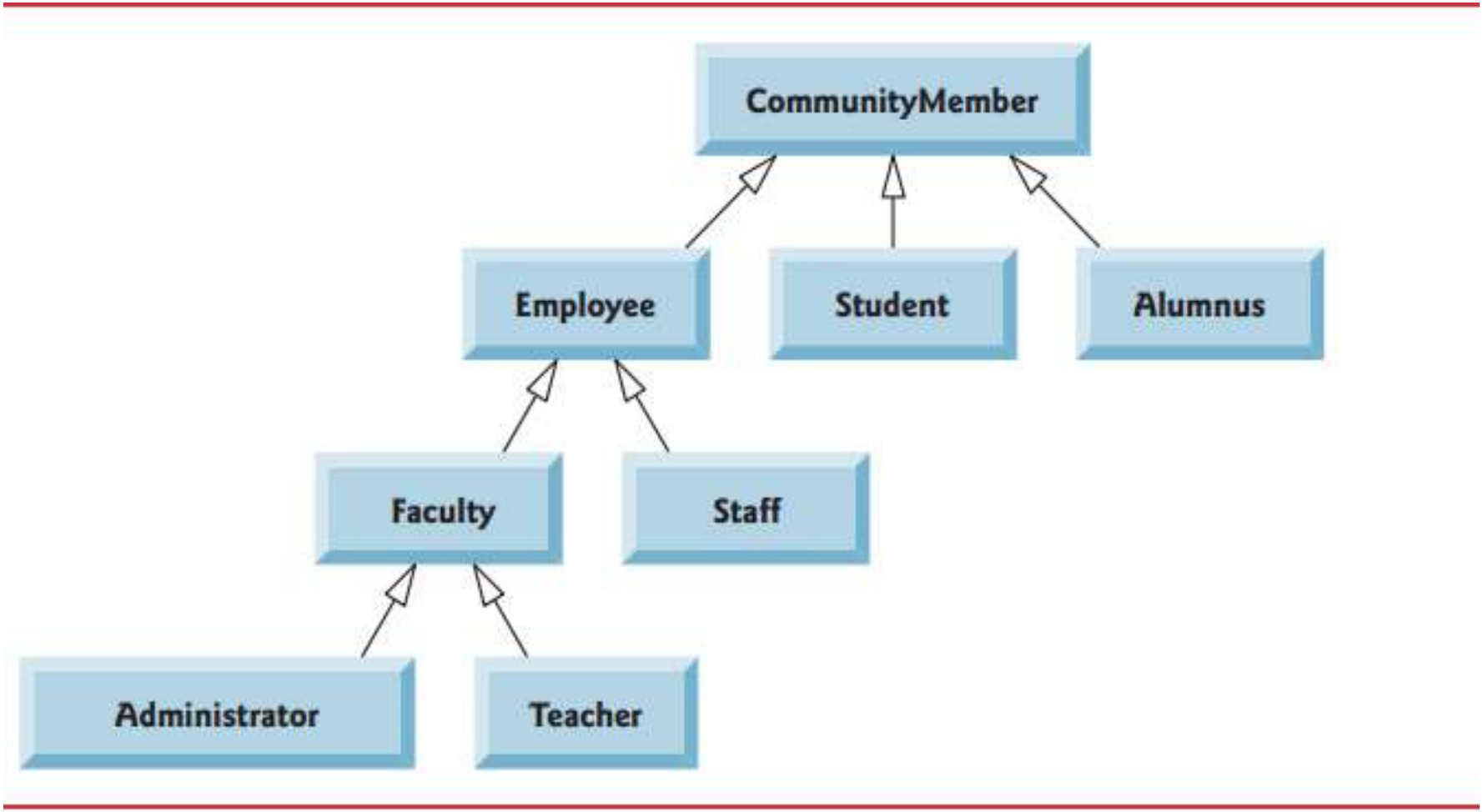


Fig. Inheritance hierarchy UML class diagram for university **CommunityMembers**

Defining a Subclass

- A subclass inherits members from a superclass.
- You can also:
  - Add new properties
  - Add new methods
  - Override / redefine the methods of the superclass with an appropriate implementation.
- Java syntax for indicating inheritance / defining subclass:

```
class SubClass extends SuperClass {
}
```

– extends keyword is used to define subclass

Using the Keyword super

- The keyword super refers to the superclass of the class in which super appears.
- Super keyword can be used in two ways:
  - To call a superclass constructor
  - To call a superclass method
- Invoking a superclass constructor's name in a subclass causes a syntax error.
- You must use the keyword super to call the superclass constructor from the subclass.

Calling Superclass Constructors

- Constructors are not inherited.
- The first task of any subclass constructor is to call its direct superclass's constructor, either explicitly or implicitly.
- Superclass's constructor is always invoked.
- The syntax to call a superclass's constructor is :

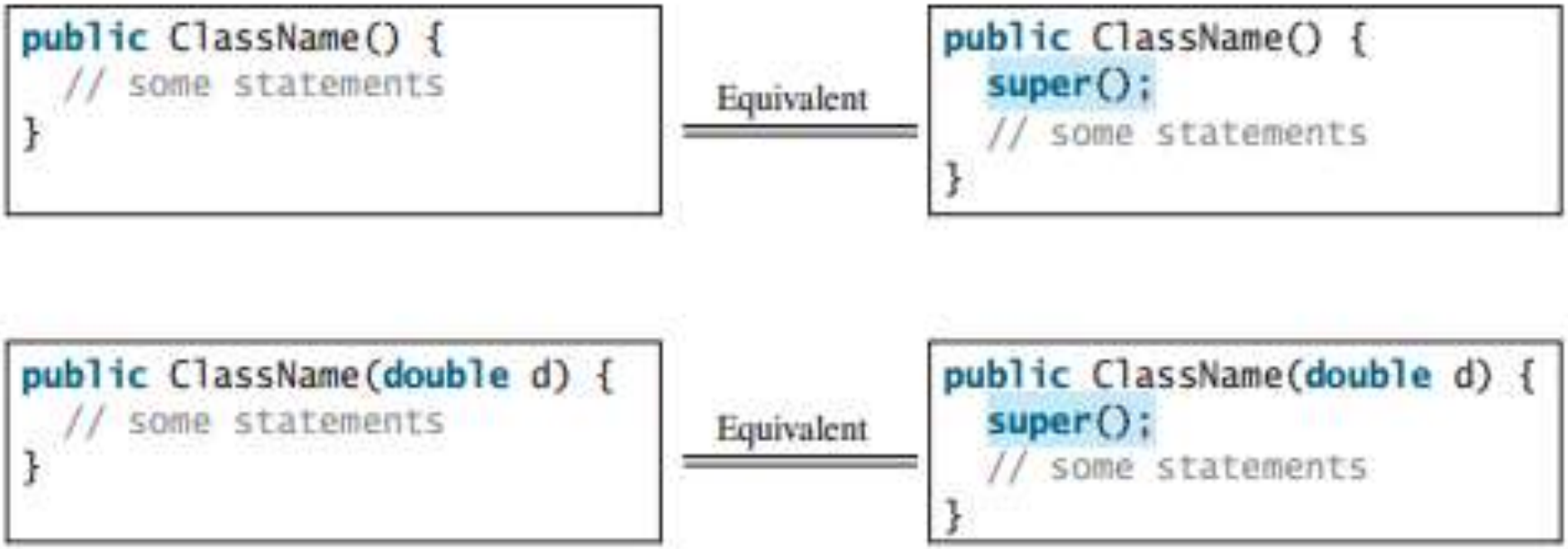
  super(), or

  super(parameters); // explicit call

- A class's default constructor (when generated by the compiler) calls the superclass's default or no-argument constructor.

Calling Superclass Constructors…

- A constructor may invoke an overloaded constructor or its superclass constructor.
- If neither is invoked explicitly, the compiler automatically puts super()as the first statement in the constructor.



Calling Superclass Constructors…

- Syntax error occurs if the superclass does not have a no-arg constructor and the subclass tries to invoke this constructor.

Consider the following code:

```
class Fruit {
  public Fruit(String name) {
      System.out.println("Fruit's constructor is invoked");
    }
}

public class Apple extends Fruit {
}
```

What is the error in this program?

Constructor Chaining

- Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain.
- This is known as ***constructor chaining***.

Example of constructor chaining

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();
  }
  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }
}
```

```
class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee’s overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
  }
  public Employee(String s) {
    System.out.println(s);
  }
}
```

```
class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }
}
```

Using Super to call superclass method

- The syntax is:

  super.method(parameters);

Example:

```
1  package chapter3;
2
3  public class Parent {
4
5      void display(){
6          System.out.println("Parent Display");
7      }
8
9  }
```

```
1  package chapter3;
2
3  public class Child extends Parent {
4
5      @Override
6      void display(){
7          System.out.println("Child Display");
8          super.display();
9      }
10 }
11
12 class Test{
13     public static void main(String[] args) {
14         Child c =new Child();
15         c.display();
16     }
17 }
18
19
```

Output???
Child Display
Parent Display

Using Super to call superclass method

- Note: it is not always necessary to use the keyword super to call superclass method when:
  - Method is not overridden
  - To call child class overridden method, without super java calls child's overridden method
- Java don't allow to use super keyword inside a static method because super refers to parent class object.
- It is acceptable to call a static method from a constructor.
- But java don't allow overridable methods to be called from constructors

Overriding Methods

- Modifying the implementation of an inherited method defined in the superclass is referred to as method overriding.
- To override a method, the method must be defined in the subclass using the same signature and the same return type as in its superclass.
- **@Override** annotation:
  - Indicates that a method should override a superclass method with the same signature.
  - If it does not, a compilation error occurs in earlier JDK versions.

Overriding Methods...

Example:

```
@Override
 public String toString()
 {
  // specific implementation
 }
```

- It's a compilation error to override a method with a *more restricted access such as private, final, etc.*

Overriding Methods...

- A private method cannot be overridden.
- A static method can be inherited, but cannot be overridden.
- If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.
- The hidden static methods can be invoked using the syntax:

SuperClassName.staticMethodName

Overriding vs. Overloading

```
public class Test{
 public static void main(String[] args){
   A a = new A();
  a.pow(10);
   a.pow(10.0);
 }
}
class  B {
 public void p(double i){
   System.out.println(i * 2);
  }
}
class A extends B
// This method overrides the method in B
 public void p(double i){
   System.out.println(i);
  }
}
```

```
public class Test{
 public static void main(String[] args){
  A a = new A();
  a.pow(10);
  a.pow(10.0);
 }
}
class  B {
 public void p(double i){
   System.out.println(i * 2);
  }
}
class A extends B
// This method overloads the method in B
  public void p(int i){
    System.out.println(i);
  }
}
```

Implementing Inheritance

- *At the design stage in an object-oriented system, you'll often find that certain classes are closely related.*
- *You should "factor out" common instance variables and methods and place them in a superclass.*
- *Then use inheritance to develop subclasses, specializing them with capabilities beyond those inherited from the superclass.*
- *Changes made to the common features in the superclass are inherited by the subclass.*

Class Object

- All classes in Java inherit directly or indirectly from **java.lang.Object** class, so its methods are inherited by all other classes.
- Commonly used Object methods are:
  - toString()
  - equals()
  - getClass()
- Learn more about Object's methods in the online API documentation.

The toString() method in Object

- The **toString()** method returns a string representation of the object.
- The default implementation returns a string consisting of a class name of which the object is an instance, the at sign (@), and a number(hash code) representing this object.

Example:

```
Loan loan = new Loan();
System.out.println(loan.toString());
```

- The code displays something like **Loan@15037e5**.
- This message is not very helpful or informative.
- Usually you should override the toString method

The equals Method

- The **equals()** method compares the contents of two objects (whether two objects are equal).

- The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {
  return (this == obj);
}
```

- The syntax for invoking **equals()**method is:

object1.equals(object2);

== vs equals()

- The == comparison operator is used for comparing two primitive data type values or for determining whether two objects have the same references.
- The equals method is intended to test whether two objects have the same contents
- The == operator is stronger than the equals method, in that the == operator checks whether the two reference variables refer to the same object.

Polymorphism

- In polymorphism a variable of a supertype can refer to a subtype object.
- Polymorphism allows to use a single variable as reference to instances of multiple subclasses of the same superclass.
- A class defines a type.
  - A type defined by a subclass is called a subtype, and a type defined by its superclass is called a supertype.

    Example: **Student** s = new **PostgradStudent**();

Therefore, you can say that **postgradStudent** is a subtype of **Student** and **Student** is a supertype for **postgradStudent**.

Polymorphism…

- The same method name and signature can cause different actions to occur, depending on the type of object on which the method is invoked.
- When a superclass variable contains a reference to a subclass object, and that reference is used to call a method, the subclass version of the method is called.

Demo

- An object of a subclass is an object of its superclass type (but not vice versa).

Dynamic Binding

- Connecting a method call to the method body is known as binding.
- When the compiler encounters a method call made through a variable, the compiler determines if the method can be called by checking the variable's class type.
  - If that class contains the proper method declaration (or inherits one), the call is compiled.
- At execution time, JVM checks the type of the object to which the variable refers to determine the actual method to use.
  - This process is called **dynamic (run time, late) binding**.

Example

```
class Person extends Object {
  public String toString() {
    return "Person";
  }
}
```

```
class GraduateStudent extends Student {
}
 class Student extends Person {
  public String toString() {
    return "Student";
  }
}
```

Example…

```
public class PolymorphismDemo {
  public static void main(String[] args) {
    m(new GraduateStudent());
    m(new Student());
    m(new Person());
    m(new Object());
  }
  public static void m(Object x) {
     System.out.println(x.toString( ) );
  }
}
```

Method **m** takes a parameter of the Object type.

You can invoke it with any object.(polymorphic call)

- When the method m(Object x) is executed, the argument x's toString method is invoked.
- x may be an instance of GraduateStudent, Student, Person, or Object.
- Classes GraduateStudent, Student, Person, and Object have their own implementation of the toString method.
- Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime -- ***dynamic binding***.

Generic Programming

- Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as generic programming.

- If a method's parameter type is a superclass (e.g., Object), you may pass an object to this method of any of the parameter's subclasses (e.g., Student or String).

- When an object (e.g., a Student object or a String object) is used in the method, the particular implementation of the method of the object that is invoked (e.g., toString) is determined dynamically.

Method Matching vs. Binding

- Matching a method signature and binding a method implementation are two issues.
- The compiler finds a matching method according to parameter type, number of parameters, and order of the parameters at compilation time.
- A method may be implemented in several subclasses.
- The Java Virtual Machine dynamically binds the implementation of the method at runtime.

Casting Objects

- *Casting* can be used to convert an object of one class type to another within an inheritance hierarchy.
- In the preceding section, the statement:

```
m(new Student());
```

assigns the object new Student() to a parameter of the Object type. This statement is equivalent to:

```
Object o = new Student(); // Implicit casting
m(o);
```

The statement Object o = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object.

Why Casting Is Necessary?

- Suppose you want to assign the object reference **o** to a variable of the Student type using the following statement:

```
Object  o = new Studnet();

Student b = o;   // A compile error would occur
```

- *Why does the statement **Object o = new Student()** work and the statement **Student b = o** doesn't?*

- This is because a Student object is always an instance of Object, but an Object is not necessarily an instance of Student.

- Even though you can see that **o** is really a Student object, the compiler is not so clever to know it.

- To tell the compiler that **o** is a Student object, use an explicit casting. The syntax is:

```
Student b = (Student)o; // Explicit casting -- downcasting
```

`instanceof` Operator and Downcasting

- Explicit casting must be used when casting an object from a superclass to a subclass.
- This type of casting may not always succeed.
- Attempting to assign a superclass reference to a subclass variable is a compilation error.
  - To avoid this error, the superclass reference must be casted (downcasting reference) to a subclass type explicitly.
- At execution time, if the object to which the reference refers is not a subclass object, a ***ClassCastException*** exception will occur.
  - Use the ***instanceof*** operator to ensure that such a cast is performed only if the object is a subclass object.

DEMO - Typecasting.java

Abstract Classes and Methods

- Abstract classes:
  - Cannot be used to instantiate objects—abstract classes are too general and *incomplete*.
  - Subclasses must declare the "missing pieces" to become "**concrete**" classes, from which you can instantiate objects; otherwise, these subclasses, too, will be abstract.
  - An abstract class declares common attributes and behaviors (both abstract and concrete) of the various classes in a class hierarchy.
  - enables these classes to inherit those attributes and behaviors and share a common design.

Abstract Classes and Methods…

- An abstract class must contain at least one *abstract* method.
- Abstract methods do not provide implementations (method bodies).
- Each concrete subclass of an *abstract* superclass must provide concrete implementations of each of the superclass's abstract methods otherwise the subclass is also declared abstract.

- **Constructors, *private*, *final,* and *static* methods cannot be declared *abstract*. Why?**

final Methods and Classes

- A *final* method cannot be overridden in a subclass.
  - *Private* and *static* methods are implicitly *final*.
  - Calls to *final* methods are always resolved at compile time - this is known as static (compiler time, early) binding.
- A final class cannot extended to create a subclass.
- All methods in a *final* class are implicitly *final*.
- A final data field is a constant.

Example: final static double PI = 3.14159;

Creating and Using Interfaces

- An interface is a class like construct that contains only constants and abstract methods.
- An interface is used to specify common behaviors (methods and constants) for objects.
- An interface is often used when disparate (unrelated) classes need to share common methods and constants.
- Allows objects of unrelated classes to be processed polymorphically by responding to the same method calls.
- You cannot create an instance from an interface using the new operator.
- Interfaces may not specify any implementation details, such as concrete method declarations and instance variables.

Creating and Using Interfaces…

- Syntax to define an interface:

```
public interface InterfaceName {
  //constant declarations and abstract method signatures
}
```

- All methods declared in an interface are implicitly *public abstract* methods.
- All data fields are implicitly *public, static,* and *final*.

For example:

```
public interface T1{
  public static final int K = 1;
   public abstract void p();
}
```

Equivalent

```
public interface T1{
      int K = 1;
      void p();
}
```

Creating and Using Interfaces…

- A constant defined in an interface can be accessed using syntax:

InterfaceName.CONSTANT_NAME ;

Example: T1.K

- To use an interface, a concrete class must specify that it implements the interface and must declare each method in the interface with the signature specified in the interface declaration.
- To specify that a class implements an interface, add the **implements** keyword and the name of the interface to the end of your class declaration's first line.
- Implementing an interface is like signing a contract with the compiler that states, "I will implement all the methods specified by the interface or I will declare my class **abstract**."
- Interfaces are Java's way to provide multiple inheritance.

Example: Using Interfaces

```
public interface Testable {
    int K = 10;
    void message();


}
```

Interface declearation //same as class declearation

```
public class Test implements Testable{

    @Override
    public void message() {

    }
   public static void main(String [] args) {


   }
}
```

A class that implement the interface should override the methods of the interface otherwise syntax error !

```
public abstract class Test2 implements Testable{

}
```

or the class that implements the interface should be abstract class.

DEMO

Interfaces vs. Abstract Classes

	Variables	**Constructors**	**Methods**
Abstract class	No restrictions	Constructors are invoked by subclasses through constructor chaining. An abstract class cannot be instantiated	No restrictions
Interface	All variables must be **public static final**	No constructors. An interface cannot be instantiated	All methods must be public abstract instance method

Functional Interfaces

- As of Java SE 8, any interface containing only one *abstract* method is known as a functional interface.
- For example:
  - ***ActionListener*** —You'll implement this interface to define a method that's called when the user clicks a button.
  - ***Comparator*** —You'll implement this interface to define a method that can compare two objects of a given type to determine whether the first object is less than, equal to or greater than the second.
  - ***Runnable*** —You'll implement this interface to define a task that may be run in parallel with other parts of your program.